

Holistically Evaluating Reasoning Over Legal Texts in Large Language Models

Sophia Moore
sophia.moore@yale.edu

Advisor: Arman Cohan
arman.cohan@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science &
Mathematics*

Department of Computer Science
Yale University
May 1, 2025

Acknowledgements

This thesis concludes a semester-long project; more importantly, it represents the culmination of many people's sustained love, support, and hard work. I owe a great deal of thanks to those who have and continue to lift me up — here, I'd like to take the time to thank a few.

This work was supported by the Yale Natural Language Processing Lab, as a subdivision of the project led by Simeng Han. I am extremely grateful to both Simeng and Arman Cohan, the leader of this lab, for their continued support and mentorship.

I would also like to express my sincere gratitude to the Yale Department of Computer Science and Yale Department of Mathematics for providing me with so many opportunities for academic, professional, and personal growth during my undergraduate experience. The lessons I learned here extend far beyond the classroom.

I could not have made it without the intergenerational support of the Yale Varsity Swimming & Diving team. Being part of this family has been the greatest honor and privilege.

None of this would have been possible without the women who encouraged me to take up space. I am indebted to Nicole Cozza, for her conviction in my competence; to Monica Ortiz, for paving the way for me; and most importantly, to Elizabeth Ribaud, for standing steadfast in my corner. They showed me — and made me believe — that a woman can do anything she sets her mind to.

As always, I owe the biggest thank you to my parents. Of all the accolades I've ever earned, the title I'm most proud of is being their daughter.

Contents

1	Introduction	5
2	Background	7
2.1	IRAC Framework	7
2.2	The SARA Dataset	8
2.3	The <i>LegalBench</i> Benchmark	10
3	Methodology	11
3.1	Models	11
3.2	Case Formulation	12
3.3	Prompting	13
3.4	Evaluation	14
4	Results	16
4.1	Common Model Errors	17
4.2	Outlier Cases	18
4.3	Parameter Count	20
4.4	Variation Across Families	21
4.5	Open-Source vs. Commercial Models	21
4.6	Comparing GPT Models	22
5	Related Work	24
5.1	Existing Legal Benchmarks	24
5.2	Natural Language and Statute Translation	24
5.3	Future Directions	25
6	Conclusions	26
7	Future Work	27
A	Appendix	29
A.1	Code and Dataset	29
A.2	Outlier Cases	29
A.3	Model Links	29

Holistically Evaluating Reasoning Over Legal Texts in Large Language Models

Sophia Moore

Abstract

Large language models (LLMs) continue to grow in both capacity and usage, particularly in the legal community. As such, scholars are working to determine whether LLMs can accurately reason over legal texts. A significant outstanding question is whether models can apply the accepted IRAC legal framework — Issue-spotting (I), rule-Recall (R), rule-Application (A), and rule-Conclusion (C) — to a set of facts.

This thesis modifies an accepted statutory reasoning assessment dataset rooted in American tax law, and it creates a new natural-language subset on which models can be prompted to apply the IRAC framework. Additionally, we present an empirical evaluation of 19 open-source and commercial LLMs in their modular application of IRAC framework components; provide an analysis of common patterns and accuracy metrics; and outline directions for future and related work.

Our results demonstrate that larger parameter counts generally correlate to higher scores across components of the IRAC framework, as well as evaluation metrics such as balanced accuracy and mean absolute percent error (MAPE). Family-specific trends appear to hold across different size points, suggesting that pretraining and instruction-tuning play an important role in driving variation amongst different model families. We notice a substantial gap between the performances of commercial and open-source models, particularly in the areas of rule-recall and rule-application. We also observe that rule-application is the lowest scoring category across all models, regardless of parameter count. Importantly, this indicates that chain-of-thought prompting could play a significant role in improving models’ analytic capabilities. Areas for future work include expanding tasks to include other categories of legal reasoning and experimenting with model fine-tuning.

1 Introduction

The increasing capacities and prominence of large language models (LLMs) lends itself to applications across various industries. In particular, adopting LLMs to the legal field could increase accessibility to legal resources and alleviate lawyer burden. The existing benchmark for evaluating an LLM’s legal reasoning capacity, *LegalBench*, has been criticized for various reasons. *LegalBench* tasks, which center around civil law, do not reflect the heterogeneous language use of the law; they also do not deal with legal ambiguity. The result is that there is no realistic standard of evaluation for longer, more complex tasks.

Notably, the *LegalBench* benchmark independently tests LLM capabilities across the four “IRAC” categories: Issue-spotting (I), rule-Recall (R), rule-Application (A) & rule-Conclusion (C). This misrepresents a model’s true capabilities, as to produce a correct answer in a real-world scenario, a model must correctly work across the four subdivisions simultaneously. This thesis represents a smaller aspect of the work conducted by Simeng Han in the Yale Natural Language Processing Lab; her project seeks to develop a more reliable legal benchmark for large language models.

The aim of this paper is to explore the ways in which we can more accurately evaluate an LLM’s ability to reason over legal texts; we do so through designing through a more comprehensive testing approach. Importantly, this paper uses the term “large language model” (LLM) to refer to models that possess the ability to learn in-context — typically, these models consist of over a billion parameters. This thesis seeks to make the following contributions:

1. Establish a subset of open-ended natural language tasks, describing the legal context and construction process. These tasks have bases in an existing, accepted statutory reasoning dataset and have been reframed to test models’ abilities to apply the aforementioned legal reasoning framework in sequential steps.
2. Provide an evaluation of 19 open-source and commercial models across 11 different model families and three parameter tiers. We make observations regarding general performance trends; we also compare differences across model families, parameter counts, closed-API and open-source models. Finally, we break down performance within the commercial model subset.
3. Highlight future directions for performance improvement via prompt-engineering and intentional legal application. We also provide references to other related studies and existing gaps in literature.

Our results demonstrate that parameter size generally correlates to higher model scores. It follows, then, that the commercial models tested (most of which are assumed to be hundreds of billions of parameters) noticeably outperform the closed-API models. We also observe that models generally perform better on binary questions (true/false) versus open-ended. Rule-application is the lowest-scoring category across all models tested, indicating that tailored future work (such as chain-of-thought-prompting) could be used to improve analysis.

The ultimate goal of this thesis is to present a mechanism to refine the existing legal benchmarks and corresponding evaluation metrics. No argument is made regarding whether language models can or should replace the work of legal professionals; instead, we aim to contribute to a greater understanding of how LLMs are able to reason using natural language and pre-existing legal frameworks.

2 Background

2.1 IRAC Framework

American legal scholars frequently employ the Issue-spotting (I), rule-Recall(R), rule-Application (A) and rule-Conclusion (C) framework to determine the legal conditions that arise from a set of events or occurrences, oftentimes referencing prior cases and/or codified laws in the process. The IRAC framework entails the sequential application of these steps. We can break down the order of operations as follows:

Issue-spotting: Lawyers identify either an explicit or implicit legal issue in a given set of facts. Frequently, an issue is either specific unanswered legal question posed by the facts, or a certain area of law implicated in the scenario.

Rule-recall: Lawyers identify the legal rules relevant to the given facts. A rule is a legal statement dictating the necessary or sufficient conditions to obtain a certain legal outcome. American legal rules can be sourced from the Constitution, federal and/or state statutes, regulations, and court opinions. Notably, rules can differ between jurisdictions.

Rule-application: Lawyers apply the aforementioned rules to the scenario. This process involves determining which facts are most relevant to the rule, as well as how those facts indicate a certain outcome. This step can involve citing precedent or prior cases that use similar rules to determine the outcome of the given scenario.

Rule-conclusion: Lawyers reach a conclusion regarding the legal outcome of the scenario through combining their findings from the previous steps.

The *LegalBench* benchmark also tests across two additional categories of reasoning:

Rhetorical Understanding: This refers to lawyers' ability to understand arguments and exercise interpretive skills to identify the rights, obligations, and/or limitations of certain legal languages.

Interpretation: This refers to the ability to parse and understand a legal text (i.e. extract a relevant piece of information, correctly answer a question, or categorize the text)

The *LegalBench* tasks evaluate each type of legal reasoning independently. In reality, however, a lawyer employing the IRAC framework would generate their final answer through sequentially building off inferences from the previous steps. To produce a more

realistic assessment, we evaluate the IRAC steps in sequence. Because rhetorical understanding and interpretation are distinct tests of an LLM’s capabilities, we exclude assessment of these two processes and categorize it as future work.

2.2 The SARA Dataset

We begin by testing models’ abilities to sequentially apply the steps of the IRAC framework to the StAtutory Reasoning Assessment (SARA) dataset. Statutory reasoning refers to the application of legislation to facts of a case. The SARA dataset was created to investigate the effects of natural language understanding approaches on statutory reasoning, since the latter implicitly requires the former. In particular, the SARA dataset consists of the following:

1. A set of rules extracted from the statutes of the United States Internal Revenue Code (IRC)
2. A set of natural language questions that can be answered correctly only through correct interpretation of the aforementioned rules.

We use the SARA dataset to systematically evaluate whether large language models can situationally apply the logic of these legal rules to real-life cases. This is particularly challenging for LLMs: oftentimes, the necessary information for creating a verdict is declared only once (i.e. in a law), whereas traditional machine reading relies on distributional language statics.

We can characterize the IRAC framework as it pertains to the SARA Dataset in the following way:

Issue-spotting: What is the legal issue(s) relevant to the task at hand?

Rule-recall: What rule from the Internal Revenue Code (IRC) applies to that issue?

Rule-application: How does the rule apply to the facts of the case?

Rule-conclusion: What conclusion can be drawn from the application?

Importantly, key features of American law retained across SARA cases include reasoning with time, exceptions and substitutes, numerical reasoning, cross-references, common sense knowledge, and hierarchical structure.

The SARA dataset itself consists of both binary prediction tasks and integer prediction tasks. In the former, the model is prompted to classify a question, given in context, as an *entailment* or *contradiction*. *Entailment* refers to an instance in which the statute applies,

and *contradiction* refers to a scenario in which it does not. An example of a binary prediction task can be seen in Figure 2.1.

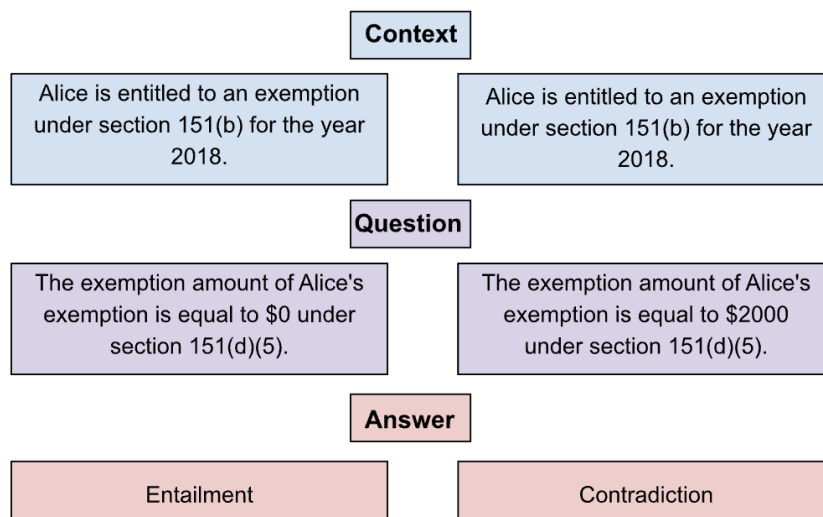


Figure 2.1: Examples of binary prediction tasks (*sara_entailment*) from the SARA dataset. Models are prompted to classify the *Question* as ‘Entailment,’ meaning it follows logically from the *Context* field, or ‘Contradiction,’ indicating that it does not.

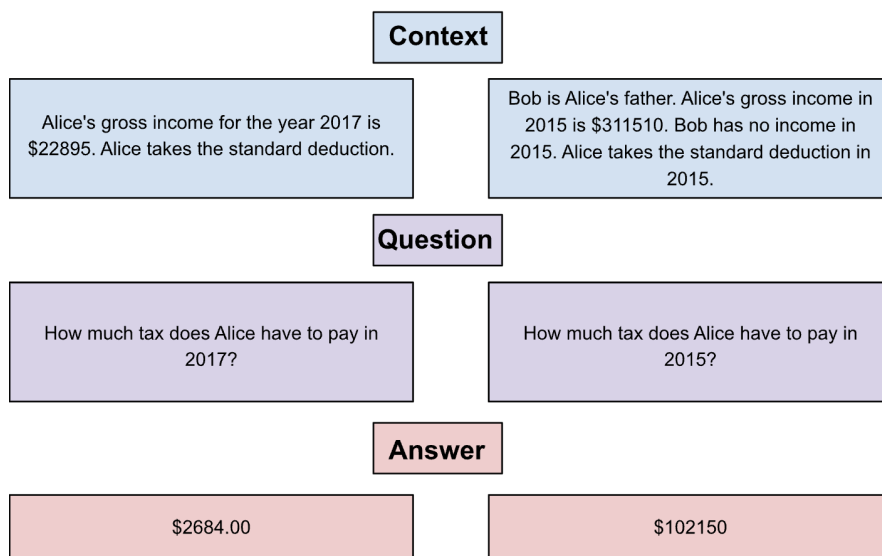


Figure 2.2: Examples of integer prediction tasks (*sara_numeric*) from the SARA dataset. Models are prompted to generate a numeric answer to the given *Question* using information contained in the *Context* field.

The SARA dataset also contains 100 integer prediction tasks, which prompt the model to take into account the entirety of the statutes. Examples of integer prediction tasks can be seen in Figure 2.2.

The SARA dataset does not provide any reference to the statutes used to correctly generate integer solutions (or the sequence in which the statutes are applied) for *sara_numeric* tasks. As such, for ease of manual evaluation surrounding rule-application, we elect to exclude this category from our task subset.

2.3 The *LegalBench* Benchmark

The *LegalBench* framework includes tasks designed to separately test components of the IRAC framework. Specifically, the benchmark includes the full set of both *sara_entailment* and *sara_numeric* tasks (referring to the classes of binary and integer prediction tasks seen in Figure 2.1 and Figure 2.2) to evaluate interpretation skills relevant to rule-application and rule-conclusion.

LegalBench contains several tasks sets designed to test the ability to state formulations for different legal rules, identify where laws are codified, and answer general questions about doctrine. These tasks are designed to test model rule-recall, and are intended to be evaluated as zero-shot tests assessing baseline model capabilities [1]. The authors used legal professionals to manually verify model outputs; manual evaluation by a law-trained individual is an established practice for works surrounding LLM function in the legal realm [1].

Finally, *LegalBench* contains various issue-spotting tasks designed to determine whether a specific scenario raises certain legal questions, implicates a specific area of the law, or is especially relevant to a certain party. 16 of the 17 tasks are drawn specifically from a public legal aid forum, and the LLM is prompted to determine whether the given post implicates a particular domain of law [1].

3 Methodology

Our experiment adapts the dataset from Holzenberg et al [2]. The modified dataset and corresponding prompt generation and run scripts can be found in the code repository linked in Appendix A.1.

Our experiment has a similar primary goal to the *LegalBench* paper: observe any performance differences across model families, comment on the role of model size, and determine whether there is a gap between open-source and commercial LLMs.

3.1 Models

We study both commercial API-access models and open-source models of different size points.

Commercial: We evaluate four commercial API-access models. Results from the following models were retrieved between March and April of 2025:

- OpenAI GPT: GPT-3.5-Turbo, GPT-4o, GPT-4
- Anthropic: Claude-3.7-Sonnet

Open Source: We evaluate 15 open-source models across three different parameter size points: 3B parameters, 7B parameters, and 13B parameters. We provide Hugging Face links for each model in Appendix A.3.

- Together: Incite-Instruct-7B, Incite-Base-7B, Incite-Instruct-3B
- Meta OPT: OPT-2.7B, OPT-6.7B, OPT-13B
- TII Falcon: Falcon-7B-Instruct
- MosaicML MPT: MPT-7B-8k-Instruct
- LMSYS Vicuna: Vicuna-7B-16k, Vicuna-13B-16k
- Google FLAN-T5: Flan-T5-XL, Flan-T5-XXL
- Wizard: WizardLM-13B
- BigScience BLOOM: BLOOM-3b, BLOOM-7B

3.2 Case Formulation

To satisfy our new requirements, we make the following changes to the natural language prompts contained in the SARA dataset:

1. **Task Selection.** Within the SARA dataset, cases are split into 176 train and 100 test samples. Each pair of positive and negative cases are included in the same split, and each section is divided between training and testing in the same proportion as the larger split. To test the models’ baseline capabilities (via zero-shot prompting), we did not initially select any samples from the available train-split.

As mentioned previously, because *sara_numeric* tasks do not cite the exact statutes (or sequence) used to generate the integer solutions, we elect to exclude them for ease of manual evaluation.

2. **Task Modification.** To test whether the model can effectively and sequentially perform components of the IRAC framework, we modify task prompts to convert them into true/false or open-ended questions. This reflects the notion that the model’s generated output should be some conclusion based on rule-recall and rule-application, rather than simply identifying a statement as an entailment or contradiction to the given context. We give an example of a *sara_entailment* task modification below:

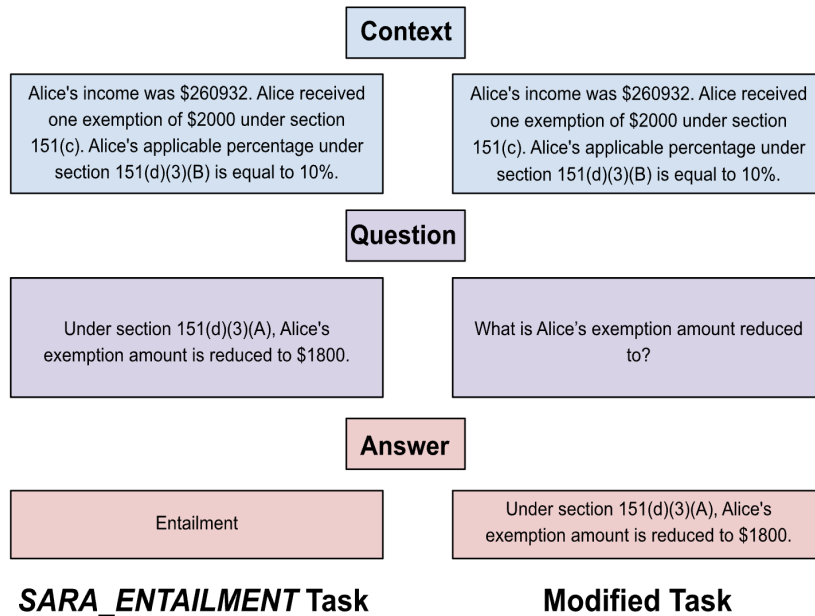


Figure 3.1: An example of the task modification process based on the SARA dataset. Tasks are converted from binary prediction tasks (identifying a statement as ‘entailment’ or ‘contradiction’) to a true/false or open-ended question requiring application of the IRAC framework.

This reframing mechanism was used across all tasks included in the subset. The *sara_entailment* tasks whose corresponding question explicitly asked whether a certain statute applied to the given context were excluded from the subset; these reformulated tasks could not test model rule-recall.

Legally, we define an issue as a specific unanswered legal question posed or a specific area of law that is implicated in a given scenario. Existing literature indicates that issue-spotting is oftentimes a narrow problem — lawyers are frequently told the issue before proceeding with a given text [1]. Because we are working specifically with a statutory reasoning dataset, we regard issue inference as trivial. Instead, we structure our tasks so that the prompt poses an explicit question (or issue), such that the model is asked to perform rule-recall, rule-application, and rule-conclusion.

The result is that we yield two task classes: one that requires integer prediction answers after successful application of a rule (such as in Figure 3.1), and one that requires a binary true or false value. We will henceforth refer to these classes as *IRAC_Binary* and *IRAC_Numeric*, respectively.

3. **Task Dating.** To reflect adaptation of the SARA tasks to a zero-shot, rule-recall setting (and adhere to the *LegalBench* precedent for rule-recall), we verify that all tasks are dated. Model prompts explicitly require use of a versioned IRC.
4. **Redundancy Reduction.** Multiple tasks in the SARA dataset include identical contexts, but differing questions. For questions that we have reframed as open-ended (whereas the previous prompts were binary), these multiple variations naturally collapse into one.

3.3 Prompting

Prompts were designed by reframing *sara_entailment* tasks as either true/false or open-ended questions. Following *LegalBench* precedent, we used the same prompts across all tested LLMs. For all models, we set a maximum generation length of 500 tokens. We used a temperature of 0.7 to introduce an element of controlled randomness — we wanted the model to explore a range of plausible completions while still favoring higher-probability outputs. These parameters were held constant to ensure consistency in both output length and generation behavior.

There is significant room for improvement in refining the prompts used in this paper. As such, the given results represent a lower-bound on model performance; higher quality prompts will induce higher scores. The given prompts should be regarded as a jumping-off point for future work.

Base prompts and prompt generation scripts can be found in the accompanying code repository linked in Appendix A.1.

3.4 Evaluation

To align with the precedent for evaluating LLM performance in legal domains, a law-trained professional manually evaluated each output for correctness. To account for differences in tense, casing, and/or punctuation, we normalized generated outputs for extraction tasks. Models were assigned a score based on the quality of their analysis. We give an example below:

Incorrect Analysis	Partially Correct Analysis	Correct Analysis
The relevant section of the Internal Revenue Code is Section 2(a), which defines a surviving spouse as someone who remains unmarried at the end of the taxable year following the death of their spouse.	Section 2(a) of the Internal Revenue Code defines a surviving spouse as someone whose spouse has died during the tax year and has not remarried before the end of the tax year.	In Section 2(a), the Internal Revenue Code defines a surviving spouse as a taxpayer whose spouse died during either of the two taxable years immediately preceding the taxable year. To qualify as a surviving spouse, the person must not have remarried.
In this scenario, Alice died on July 9th, 2021, which means Bob was a surviving spouse in 2020, since he was still married to Alice at that time.	In this scenario, Alice died on July 9th, 2014, and Bob did not remarry by the end of 2016. Therefore, Bob was a surviving spouse in 2016.	In this scenario, Alice died on July 9th, 2014, and Bob did not remarry by the end of 2016. Therefore, Bob was a surviving spouse in 2016.

Figure 3.2: An example of how different model outputs are evaluated for correctness

In Figure 3.2, we have three example model outputs. In the leftmost column, the model incorrectly applies the definition of a surviving spouse to Bob. Clearly, because Alice was still alive in 2020, Bob cannot be labeled a surviving spouse. In the middle column, the model correctly recalls one aspect of the definition of a surviving spouse (someone whose spouse died during the current tax year). Though the model eventually comes to the correct conclusion, it fails to apply the correct component of Section 2(a) to the scenario: the model should cite that Alice died in the two taxable years immediately preceding the taxable year, and so Bob is a surviving spouse in 2016. The rightmost column shows a correct example of rule-recall and rule-application (thus, it represents a correct example of rule-conclusion).

We specifically use the following metrics to quantify rule-conclusion accuracy:

IRAC_Binary: Classification tasks are evaluated using an “exact-match” (following HELM [3]). To account for the label imbalance, we use the balanced-accuracy equation as a metric:

$$\text{Balanced Accuracy} = \frac{1}{N} \sum_{i=1}^N \frac{TP_i}{TP_i + FN_i}$$

Where:

- TP_i = True positives for class i
- FN_i = False negatives for class i

- N_i = Total number of classes

We report this metric as part of the evaluation of the rule-conclusion category for model performance on *IRAC_Binary* tasks. At a high level, this formula gives equal weight to each class, thus providing a more reliable representation of model performance.

IRAC_Numeric: Though this is not the central focus, we report the mean absolute percent error (MAPE) across the model’s numerical predictions. We do this in addition to reporting the raw percentage of correct predictions for some simplified comparison metric:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100, \quad \text{for } y_i \neq 0$$

This metric is included as part of the evaluation of the rule-conclusion category for model performance on *IRAC_Numeric* tasks. This metric only represents the average absolute error across questions for which the model successfully returns a numeric answer.

4 Results

The results of our experiment can be visualized in the following table:

3B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Bloom-3B	7.97	3.26	13.77	16.53	684.014%
Flan-T5-XL	5.80	<u>4.35</u>	<u>47.26</u>	<u>65.69</u>	<u>226.39%</u>
Incite-Instruct-3B	<u>10.87</u>	<u>4.35</u>	39.13	43.76	261.11%
Opt-2.7B	2.90	2.17	34.42	49.18	518.86%
7B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Bloom-7B	13.68	6.16	23.91	31.33	178.68%
Falcon-7B-Instruct	4.71	2.90	35.50	42.74	227.81%
Incite-7B-Base	15.58	7.25	14.49	12.12	<u>48.34%</u>
Incite-7B-Instruct	<u>30.07</u>	7.25	14.49	20.14	92.02%
MPT-7B-8k-Instruct	5.07	7.61	<u>46.38</u>	<u>64.17</u>	58.17%
OPT-6.7B	5.80	3.62	13.77	14.39	247.39%
Vicuna-7B-16k	19.20	<u>14.86</u>	39.13	52.22	149.01%
13B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Flan-T5-XXL	<u>23.55</u>	13.77	<u>50.72</u>	<u>68.72</u>	198.21%
OPT-13B	9.42	4.34	23.19	28.43	199.19%
Vicuna-13B-16k	23.19	<u>18.84</u>	31.88	44.74	<u>31.19%</u>
WizardLM-13B	7.61	10.14	38.41	41.49	33.74%
Commercial					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Claude-3.7	<u>93.48</u>	<u>71.01</u>	62.32	78.97	24.34%
GPT-4	73.19	62.32	63.04	76.87	<u>9.23%</u>
GPT-4o	74.64	66.00	<u>65.22</u>	<u>81.42</u>	29.85%
GPT-3.5-Turbo	50.72	42.75	52.90	62.79	18.22%

Figure 4.1: An evaluation of LLMs across three IRAC framework components, organized by parameter count.

The first block contains models in the 2B-3B range; the second contains 6B-7B; and the third displays 11B-13B. We underline the best performing model in each category within the block.

4.1 Common Model Errors

In this section, we outline some common model errors found in our results.

Arithmetic. Models can fall susceptible to arithmetic mistakes or incorrect mathematical comparisons. For example:

*Alice has an exemption of \$2000, and it is reduced by 10%. $2000 * .10 = 20$.*

Though the logic of this problem is set up correctly, the model ultimately produces an incorrect answer due to a calculation error.

Analysis. We notice that models occasionally identify the correct IRC statute but apply its contents incorrectly. For example:

The relevant section of the Internal Revenue Code is Section 2(a), which defines a surviving spouse as someone who remains unmarried at the end of the taxable year following the death of their spouse.

In this scenario, Alice died on July 9th, 2021, which means Bob was a surviving spouse in 2020, since he was still married to Alice at that time.

In this example, GPT-3.5-Turbo correctly cites IRC 2(a) as the relevant statute, but incorrectly struggles with the definition of a surviving spouse.

Rule Mismatch. Models can hallucinate an incorrect IRC statute but correctly apply the contents of a different rule. For example, on one task, we had GPT-3.5-Turbo generate the following response:

Section 151(c) of the Internal Revenue Code allows for an additional exemption amount for blind individuals. Since Alice has been blind since October 4, 2013, she would qualify for an additional exemption amount in 2017.

Though GPT-3.5 correctly cites and applies the fact that the IRC allows for an additional exemption amount for blind individuals, this information can actually be found in Section 63(f)(2).

Time. We notice that models can frequently confuse the chronology of events. Observe how the following statement is incorrect:

Alice and Bob have a son, Charlie. In 2017, Charlie filed a joint return with his spouse whom he married on Dec 1st, 2016.

In 2017, Charlie meets the criteria to be considered a qualifying child as outlined in Section 152(c). So, Charlie is a dependent of Bob for 2017.

A stipulation of the definition outlined in Section 152(a) is that the individual has not filed a joint return with their spouse for the that calendar year. Since Charlie filed a joint return in 2017, he cannot be considered Bob’s dependent. In this instance, the model fails to reason with time and correctly apply the statute.

In-Context Definitions. Occasionally, we notice that models confuse a term’s legal definition with its generic definition. An example of such confusion is demonstrated here:

Alice and Bob have been dating for three years. Alice and Bob are in a relationship.

While Alice and Bob can be colloquially referred to as in a “relationship,” as given, they do not meet the criteria of a legal “relationship.” More information would be needed to determine if there exists a legal relationship between Alice and Bob.

Term Confusion. We also notice that models are susceptible to term confusion and hallucination surrounding given values. For example:

In 2016, Alice’s income was \$567192. Alice is a head of household for the year 2016. Her applicable amount for 2016 is \$567192.

Here, the model mistakes Alice’s income as her applicable amount. We notice that models frequently cite a number given in the question as the answer for a separate variable.

4.2 Outlier Cases

Of our 69 tasks, we identify three *IRAC-Numeric* tasks with exceptionally poor model performance. These questions cite various subsections of IRC Statute 68, which pertains to overall limitation on itemized deductions. We elaborate more on these cases in Appendix A.2.

Removing these three tasks yields the following result table:

3B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Bloom-3B	8.33	3.41	14.39	16.53	684.01%
Flan-T5-XL	6.06	<u>4.55</u>	<u>50.00</u>	<u>65.69</u>	<u>226.39%</u>
Incite-Instruct-3B	<u>11.36</u>	<u>4.55</u>	40.90	43.76	261.11
Opt-2.7B	3.03	2.27	35.98	49.18	601.38%
7B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Bloom-7B	14.39	6.54	25.00	31.33	158.82%
Falcon-7B-Instruct	4.92	3.03	37.12	42.74	252.77%
Incite-7B-Base	16.29	7.58	15.15	12.12	<u>36.22%</u>
Incite-7B-Instruct	<u>31.44</u>	7.58	15.15	20.14	92.02%
MPT-7B-8k-Instruct	5.30	7.95	<u>48.48</u>	<u>64.17</u>	67.71%
OPT-6.7B	6.06	3.79	14.39	14.39	247.39%
Vicuna-7B-16k	20.07	<u>15.53</u>	40.90	52.22	202.40%
13B Parameters					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Flan-T5-XXL	<u>24.62</u>	14.39	<u>53.03</u>	<u>68.72</u>	183.97%
OPT-13B	7.95	4.55	24.24	28.43	230.17%
Vicuna-13B-16k	24.24	<u>19.70</u>	33.33	44.74	31.19%
WizardLM-13B	7.95	10.60	40.15	41.49	<u>18.22%</u>
Commercial					
	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
Claude-3.7	<u>97.73</u>	<u>74.24</u>	62.32	78.97	22.12%
GPT-4	76.52	62.32	65.90	76.87	<u>10.57%</u>
GPT-4o	78.03	69.70	<u>68.18</u>	<u>81.42</u>	16.21%
GPT-3.5-Turbo	53.03	44.70	55.30	62.79	12.75%

Figure 4.2: An average of LLM scores in each parameter-count block without the aforementioned “outlier” cases.

Though most models show some improvement in their rule-recall, rule-application, and rule-conclusion scores, the most noticeable difference occurs in the mean absolute percent error (MAPE) category. Removal of these cases standardizes MAPE within each parameter size grouping such that commercial models (which have the largest parameter count) display the smallest average error and the 3B-models (with the smallest number of parameters) produce the largest error margin. The MAPE scores corresponding to a few models, however, actually inflate with removal of these outlier cases.

On average, models perform far better on *IRAC_Binary* tasks than *IRAC_Numeric*.

This could be attributed in part to probability — a model has a higher likelihood of answering a true/false question correctly than an open-ended one. Existing literature suggests that chain-of-thought prompting could be used to improve models’ reasoning abilities, and in particular, it could drive improvement for models with larger parameter counts [4]. In guiding models to set up the proper framework for a problem, we could potentially induce better performance on *IRAC_Numeric* tasks. Refer to Section 7 for more information surrounding avenues for future work.

4.3 Parameter Count

From Figure 4.1, we calculate the averages across the four parameter-count blocks:

	Rule-Recall	Rule-Application	Rule-Conclusion	Balanced Accuracy	MAPE
3B	6.88	3.53	33.64	43.79	422.59%
7B	13.44	7.09	26.81	33.87	143.06%
13B	15.94	11.77	36.05	45.84	115.58%
Commercial	<u>73.01</u>	<u>60.52</u>	<u>60.87</u>	<u>75.01</u>	<u>63.86%</u>

Figure 4.3: An average of LLM scores in each parameter-count block.

As evident in Figure 4.3, commercial models display the highest average values across all three IRAC categories; they also possess the best average balanced accuracy and MAPE scores. In fact, for rule-recall, rule-application, and MAPE, parameter size directly correlates to performance. For each of these categories, commercial models perform the best on average, while the smallest models (3B parameters) perform the worst.

In examining Figure 4.1, parameter count clearly correlates to higher model scores across the first two IRAC categories. The rule-conclusion category does not necessarily follow this trend. Of note, higher parameter models more frequently gave correct rule-recall. When this was coupled with an incorrect analysis, high-parameter models tended to give incorrect rule-conclusion answers. More frequently, lower-parameter models tended to give an incorrect rule-recall (as well as incorrect rule-application), citing an incorrect basis for their rule-conclusion answers. In this way, correct conclusion was not necessarily reflective of correct reasoning; we should examine the rule-conclusion metric in the context of both rule-recall and rule-application.

This thesis echoes the findings of Guha et al. in that we notice larger models tending to outperform smaller ones within a given LLM family [1]. Bloom-7B, for example, outscores Bloom-3B by 5.7 points on rule-recall, 2.9 points on rule-application, and 10.1 points on rule-conclusion. This is not a strict observation, however: Opt-6.7B outperforms Opt-3B in rule-recall and rule-application but not rule-conclusion.

In line with Guha et al., we also notice that the margin of difference varies across both LLM families and IRAC categories. Incite-7B-Instruct, for example, outperforms Incite-3B-Instruct by 19.2 points on rule-recall, compared to a 2.9-point difference between Bloom-7B and Bloom-3B.

Notably, the four largest models — the commercial models — possess the four highest scores across all three IRAC categories. The largest model, Claude-3.7, has the highest average score across rule-recall, rule-application, and rule-conclusion; likewise, the fourth-largest model, GPT-3.5-Turbo, has the fourth-highest average score.

As seen in Figure 4.3, the average model performance tends to increase with parameter count. This observation is in-line with previous findings surrounding the impact of model size on reasoning capabilities [4].

4.4 Variation Across Families

As evident in Figure 4.1, parameter size is generally indicative of model performance. Within a singular parameter-count block, however, we find substantial differences in performance. For example, Incite-Instruct-3B outperforms Bloom-3B across all three reasoning categories, in addition to the two evaluative metrics (balanced accuracy and MAPE). This finding agrees with existing literature: a model’s pretraining data, instruction-tuning mechanism, and architecture will influence its performance [1].

It is likely that better-performing models possess traits that ultimately make them better suited to natural language and/or statutory reasoning. A demonstration of the effects of instruction-tuning can be seen in comparing Incite-7B-Instruct to Incite-7B-Base. Incite-7B-Instruct scores 15.15 points better in Rule-Recall and 8.02 points in balanced accuracy; it scores the same as Incite-7B-Base in rule-application and rule-conclusion. The observation that instruction-tuning can improve a model’s ability to use components the IRAC framework could be valuable for future work in the legal domain.

We also agree with the *LegalBench* finding that family-specific trends appear to hold across different size points [1]. We find, for example, a performance gap between models in Google’s Flan-T5 family and Meta’s OPT family regardless of parameter count (3B or 13B).

4.5 Open-Source vs. Commercial Models

Our findings do not agree with the *LegalBench* assertion that open-source models are capable of performance that matches or exceeds certain commercial models. We find that commercial models possess higher scores across every component of the IRAC framework (rule-recall, rule-application, and rule-conclusion).

Some open-source models, however, possess higher balanced accuracy scores. Both Flan-T5-XXL and MPT-7B-8K-Instruct possess higher balanced accuracy scores than GPT-3.5-Turbo, despite GPT-3.5-Turbo having a marginally higher rule-conclusion score. Additionally, when the “outlier” cases are removed, WizardLM-13 has a lower MAPE than Claude-3.7; this could, however, be confounded by our small task subset size.

As evident in Figure 4.1, the largest gap between commercial and open-source models exists in the rule-recall and rule-application categories.

4.6 Comparing GPT Models

We provide a more detailed breakdown of commercial model performance with the hope that it will illustrate strengths and pitfalls of their legal applications. Commercial models possess the highest parameter count of our tested models, correlating to higher scores across rule-recall, rule-application, and rule-conclusion (in addition to balanced accuracy and MAPE).

- **Rule-Recall:** In comparing rule-recall scores across the four commercial models, we notice that Claude-1 dramatically outperforms GPT-3.5-Turbo, GPT-4o, and GPT-4 across our set of 69 tasks. In fact, using absolute terms, rule-recall tasks represent the largest performance gap between Claude-3.7 and the other closed-API models: the largest margin of difference is roughly 42.8 points (with GPT-3.5-Turbo), and the smallest is approximately 18.8 points (with GPT-4o).

A notable contributing factor to GPT-3.5-Turbo’s lag behind was its failure to consistently identify specific sections of the IRC (for example, pointing generally to Section 152 rather than rule 152(b)). Though Claude-3.7 occasionally also lacked specificity in its rule-recall, it only misattributed an IRC statute on one task.

- **Rule-Application:** Following the general trend, across all four commercial models, rule-application tends to be relatively low-scoring. We notice various instances in which models would recall the correct rule and misapply it to the given facts — refer to Section 4.1 for an overview of common model errors or Figure 3.2 for an example of incorrect analysis.

Once again, the largest margin of difference is between Claude-3.7 and GPT-3.5-Turbo (roughly 28.3 points), and the smallest gap is between Claude 3.7 and GPT-4o (7.1 points).

- **Rule-Conclusion:** In terms of rule-conclusion, we notice that GPT-4o slightly outperforms GPT-4 (2.2 points) and Claude-3.7 (2.9 points). Of note, Claude-3.7 elected to omit an answer (or not give a “direct” response) to certain tasks prompts. This slight difference could potentially be attributed to a small sample size.

Producing a correct conclusion requires significantly less exercise of model ability than rule-recall or rule-application; for *IRAC_Binary* tasks, there is a fifty percent

chance that the model guesses the right answer. As mentioned previously, we elect not to categorize correct rule-conclusion as dependent on successful rule-recall and rule-application in our evaluation. We hope that the results provide insight into the type and frequency of errors models make in “hallucinating” a correct answer without the corresponding train of thought. More information surrounding common model errors can be found in Section 4.1.

- **Balanced Accuracy:** The balanced accuracy score for GPT-4o slightly exceeds that of Claude 3.7 (2.5 points) and GPT-4 (4.5 points). Of note, Claude-3.7 has a higher balanced-accuracy score than GPT-4 (2 points) despite having a lower rule-conclusion score (.7 points). Though this difference is relatively small, it hints that Claude-3.7 performs better on both majority and minority classes.
- **Mean Absolute Percent Error (MAPE):** We notice that the MAPE is generally lower among commercial models than the examined open-source models. GPT-3.5-Turbo displays an abnormally high percentage error when including the aforementioned “outlier cases,” (refer to Section 4.2), however, upon removal, the model’s MAPE reduces by almost 94%.

5 Related Work

5.1 Existing Legal Benchmarks

The most prominent legal benchmark that this paper cites is the open-source *LegalBench* benchmark by Guha et al [1]. This benchmark represents an ongoing effort to standardize model evaluation in the legal domain. As previously noted, each type of task only tests one aspect of the accepted legal reasoning framework (IRAC). Other areas for improvement include dealing with ambiguity, adapting to heterogeneous law language, and expanding focus from civil law. This would allow for a more comprehensive testing standard for longer, more complex tasks.

While our work centers around application of law to a given scenario, a substantial number of LLM evaluation papers focus on standardized tests or equivalent exam questions. Examples include measuring LLM performance on law exams [5, 6]. Additionally, most existing benchmarking initiatives center around supervised training and/or fine-tuning mechanisms rather than zero- or few-shot prompting.

Benefits of working in a few- or zero- shot context (such as with our paper or with *LegalBench*) include avoiding expensive assembly of large training sets; this represents a pivot from traditional natural language processing methods that consume large amounts of training data and are evaluated over a small test set. This trade-off also allows us to produce a larger and more diverse task dataset.

5.2 Natural Language and Statute Translation

As mentioned previously, the SARA Dataset provides a natural language dataset intended to test models' abilities to perform statutory reasoning [2]. The *LegalBench* task set objectives differ from those of this paper in that the four components of IRAC are tested independently (rather than in sequence).

In addition to the John Hopkins paper translating Internal Revenue Code (IRC) statutes into a set of natural language tasks, there have been numerous initiatives to convert legal statutes into expert systems. In particular, there have been efforts surrounding corporate reorganization law [7], translation of a British Act of Parliament into base rules [8] and conversion of a Canadian Act of Parliament into a Prolog program [9]. These papers,

however, do not include a corresponding dataset or applicable program tasks.

Several other papers focus on the ability of models to reason over natural language. Notably, one 2019 study from Clark et al. poses science exam questions to models, evaluating their natural language reasoning techniques [10]. Though some of their methods — for example, contextual language modeling — could be used to improve LLM statutory reasoning, scientific reasoning differs in nature from reasoning over legal texts. In particular, backing for scientific reasoning can be drawn from a variety of sources, whereas statutory reasoning requires directly drawing from relevant legislation. This makes statutory reasoning uniquely reliant on natural language (particularly, that which is used in the legal domain).

5.3 Future Directions

As discussed in Section 7, which outlines future work, a potential avenue for improving quality of analysis could include chain-of-thought prompting. In particular, Wei et al. has shown that chain-of-thought prompting can significantly improve model performance on arithmetic, common-sense, and symbolic reasoning tasks; this effect is more pronounced for larger models [4]. This could be of significant consequence for improving reasoning over legal texts (in particular, the analysis implicit in rule-application) for commercial models, as most of these are hundreds of billions of parameters.

6 Conclusions

The goal of this paper is to present a subset of open-ended natural language questions that can be used to evaluate the ability of LLMs to reason over legal texts using the established IRAC framework. Our results show that, generally speaking, higher parameter size correlates to higher scores across all components of the IRAC Framework. Pretraining and instruction-tuning play an important role in driving result variation amongst different model families, suggesting that chain-of-thought prompting could play a significant role in increasing model performance [4].

We notice a substantial gap between the performances of commercial and open-source models, particularly in the areas of rule-recall and rule-application; rule-application is also the lowest-scoring category across all models. All tested LLMs seem to perform better on the given binary tasks, as opposed to tasks requiring integer generation.

Ultimately, we hope that our evaluation provides an overview of the impact that factors such as parameter count, pretraining, and model family have on reasoning over legal texts. We hope that this paper represents a stepping stone in unlocking the potential of LLMs in the legal domain, especially as it pertains to resolving common model errors.

7 Future Work

The models included in this paper constitute only a small portion of existing LLMs. In particular, we elect to test only general-purpose models: we do not test any LLMs trained specifically for legal tasks. Expanding this work to specialized models could represent an interesting next step and potential means to witness performance improvement.

Additionally, we do not test any open-source models larger than 13B parameters. Notably, both existing metrics and the results of this paper indicate that higher parameter counts drive better performance [11]. Including larger-parameter models represents an additional mechanism for producing more accurate model outputs.

A notable constraint of our work is the limitation of tasks to statutory reasoning, specifically surrounding United States tax law. More robust testing would include a variety of legal domains. Furthermore, United States tax law is constantly evolving. The tasks in our dataset as currently structured — as well as their answers — may not be accurate for future iterations of the Internal Revenue Code (IRC).

Though rule-recall tests implicitly require zero-shot prompting [1], one- or few- shot prompting could be used to improve model correctness and analysis across the other evaluation categories. Experiments surrounding model fine-tuning and chain-of-thought-prompting could be used to improve models’ abilities to reason over legal text after identifying the relevant rule.

Bibliography

- [1] Neel Guha et al. “LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2308.11462* (2023). arXiv: Cs.CL. URL: <http://arxiv.org/abs/2308.11462>.
- [2] Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. “A Dataset for Statutory Reasoning in Tax Law Entailment and Question Answering”. In: *Proceedings of the 2020 Natural Legal Language Processing (NLLP) Workshop*. San Diego, United States: Association for Computational Linguistics, Aug. 24, 2020.
- [3] Percy Lian et al. “Holistic evaluation of language models”. In: *arXiv preprint* (2022). arXiv: Cs.CL. URL: <https://arxiv.org/abs/2211.09110>.
- [4] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2201.11903* (2023). arXiv: Cs.CL. URL: <http://arxiv.org/abs/2201.11903>.
- [5] Martin Daniel Katz et al. “GPT-4 passes the bar exam”. In: *Available at SSRN 4389233* (2023).
- [6] Jonathan H Choi et al. “Chatgpt goes to law school”. In: *Available at SSRN* (2023).
- [7] L Thorne McCarty. “Reflections on TAXMAN: An experiment in artificial intelligence and legal reasoning”. In: *Harvard Law Review* 90, 837 (1976).
- [8] Marek J Sergot et al. “The British Nationality Act as a logic program”. In: *Commun. ACM* 29, 5, 370-386 (1986).
- [9] David M Sherman. “A Prolog model of the income tax act of Canada. In Proceedings of the 1st international conference on Artificial intelligence and law”. In: *Proceedings of the 1st international conference on Artificial intelligence and law*. 127–136 (1987).
- [10] Peter Clark et al. “From ‘F’ to ‘A’ on the NY Regents Science Exams: An Overview of the Aristo Project”. In: *arXiv preprint* (2019). URL: [arXiv:1909.01958](https://arxiv.org/abs/1909.01958).
- [11] Clémentine Fourrier et al. “Open LLM Leaderboard v2”. In: *Hugging Face* (2022). URL: <https://huggingface.co/spaces/open-llm-leaderboard/open-llm-leaderboard>.

A Appendix

A.1 Code and Dataset

The dataset will be given in the paper directory and can be found at the linked GitHub Repository. Prompts and run scripts can also be found there.

For readability, we include a .csv file containing generated test cases and sample correct outputs. Cases can be converted into batch prompts via the accompanying script. Refer to the repository’s README file for more information.

A.2 Outlier Cases

The cases described in Section 4.2 were identified as correlating to higher-than-average model error. Evidently, however, excluding these cases from our evaluation did not reduce MAPE values for all models (refer to Figure 4.2). We provide metrics excluding these tasks in hopes of providing a more level frame of reference, and we denote each of these cases with a (*) on the accompanying .csv file.

Though we do not identify a concrete reason for poor model performance on these tasks, we do acknowledge that these three *IRAC_Numeric* tasks closely resemble the open-endedness of the *sara_numeric* tasks originally excluded from the task subset. Existing literature dates the statute on which these tasks center, Section 68 of the Internal Revenue Code, to 1993; this represents a relatively recent piece of legislation. Limited documentation could contribute to lower-than-average model performance.

A.3 Model Links

All results were retrieved between April and March of 2025; all open-source models were run on A100 GPUs. The following table includes Hugging Face links for the open-source

models included in this thesis:

3B Parameters	
Bloom-3B	bigscience/bloom-3b
Flan-T5-XL	google/flan-t5-xl
Incite-Instruct-3B	togethercomputer/RedPajama-INCITE-Instruct-3B-v1
Opt-2.7B	facebook/opt-2.7b
7B Parameters	
Bloom-7B	bigscience/bloom-7b
Falcon-7B-Instruct	tiiuae/falcon-7b-instruct
Incite-7B-Base	togethercomputer/RedPajama-INCITE-Base-7B-v0.1
Incite-7B-Instruct	togethercomputer/RedPajama-INCITE-Instruct-7B-v0.1
MPT-7B-8k-Instruct	mosaicml/mpt-7b-instruct
OPT-6.7B	facebook/opt-6.7b
Vicuna-7B-16k	lmsys/vicuna-7b-v1.5-16k
13B Parameters	
Flan-T5-XXL	google/flan-t5-xxl
OPT-13B	facebook/opt-13b
Vicuna-13B-16k	lmsys/vicuna-13b-v1.5-16k
WizardLM-13B	WizardLM/WizardLM-13B-V1.2

Figure A.1: Hugging Face links for open source models